

I'll now expand the 90-day VLSI + 20 LPA preparation plan into a structured document (with sections, tables, and clear daily tasks) that you can save/print as PDF from your side.

Here is the full text you can directly paste into Word/Google Docs and export as PDF:

90-Day VLSI + 20 LPA Job Roadmap

For 6th Sem VTU ECE Student – Government Engineering College, Mosalehosahalli

1. Profile & Goal

- **Student:** 6th semester ECE, VTU 2022 scheme
- **College:** Government Engineering College, Mosalehosahalli (MH Halli)
- **Target:**
 - Strong VLSI (front-end + basic back-end) skills
 - 2–3 solid GitHub projects
 - NPTEL/online VLSI certification
 - Ready for **15–20 LPA** core roles by 2027 (off-campus + internships)
- **Daily Study Time:**
 - Weekdays: 2 hours (after college)
 - Weekends: 4–5 hours (flexible)

2. High-Level Plan (90 Days)

Phase 1 (Days 1–15): Digital & Verilog Foundations

Phase 2 (Days 16–30): Synthesis, Timing, & Lab Alignment

Phase 3 (Days 31–45): Advanced RTL & Mini Projects

Phase 4 (Days 46–60): Verification + STA (Interview Core)

Phase 5 (Days 61–75): Backend Intro + Portfolio

Phase 6 (Days 76–90): Interview Prep + Applications

Each day = **1–2 main tasks + output** (code, notes, quiz, or application).

3. Phase 1 (Days 1–15): Digital & Verilog Foundations

Objectives

- Refresh **Digital Electronics** (gates, K-maps, FFs, FSMs).
- Learn **Verilog** syntax and basic simulation.

- Complete first **lab-style experiments** and start GitHub.

Detailed Day-by-Day

Day 1 – Digital Refresh 1

- Revise: Logic gates, Boolean algebra, K-maps (2–4 variables).
- Source: Your 3rd sem notes + any NPTEL “Digital Circuits” video (30–45 min).
- Output: 1 page notes:
 - Canonical forms (SOP/POS)
 - 3 examples simplified with K-maps.

Day 2 – Digital Refresh 2

- Revise: Flip-flops (SR, JK, D, T), registers, counters (async/sync).
- Draw timing diagrams for D and JK FF.
- Output: 1 page summary + 2 timing diagrams.

Day 3 – Verilog Basics 1

- Install/confirm:
 - Icarus Verilog + GTKWave **or** use EDA Playground (online).
- Learn: module, input/output, assign.
- Code: 3 modules – AND, OR, 2:1 MUX.
- Output: Run simulations and save screenshots (or waveforms).

Day 4 – Verilog Basics 2

- Learn: always @(*), blocking (=) vs non-blocking (<=) assignments.
- Code: 1-bit full adder (dataflow + behavioral).
- Output: Waveform for all 8 input combinations.

Day 5 – Combinational Circuits

- Code: 4:1 mux, 3:8 decoder, 8:3 encoder.
- Write simple **testbenches** (initial block, stimulus).
- Output: Working testbenches for all three modules.

Day 6 – Sequential Basics

- Learn: always @(posedge clk) usage.
- Code:
 - D flip-flop
 - 4-bit shift register

- Output: Simulate with clock; observe transitions.

Day 7 – Mini-Review & Quiz

- Revise code from Days 3–6.
- Attempt ~20 MCQs on digital + Verilog (GATE/online).
- Output: List of questions you got wrong and corrected answers.

Day 8 – Counters

- Code:
 - 4-bit synchronous up counter
 - 4-bit mod-10 counter
- Output: Simulations; note count sequence.

Day 9 – FSM Basics

- Understand: Mealy vs Moore, state diagrams.
- Design: Sequence detector (e.g., detect “1011”).
- Code: FSM in Verilog + testbench.
- Output: Waveforms showing detection.

Day 10 – Lab Alignment 1 (Adder/Subtractor)

- Map to VTU VLSI lab expt (4-bit adder/subtractor).
- Add: Overflow flag, carry out.
- Output: Verified module + testbench.

Day 11 – Lab Alignment 2 (Latches/FFs)

- Code: SR latch, D latch using gates, then using always block.
- Output: Timing diagrams, note metastability concept.

Day 12 – GitHub Setup

- Create GitHub account (if not already).
- Create repo: vlsi-learning-journey.
- Push: All Verilog files + a README.md (short description).
- Output: Public repo link.

Day 13 – Verilog Practice Day

- Pick 3 small problems: parity generator, comparator, priority encoder.
- Code + testbench.
- Output: Verified designs in repo.

Day 14 – Weekly Consolidation

- Reorganize code in folders: combinational, sequential, fsm.
- Document: What you learned, what is still confusing.
- Output: WEEK1_SUMMARY.md in GitHub.

Day 15 – Mock Viva (Digital + Verilog)

- Prepare answers for:
 - Difference between blocking and non-blocking.
 - Synthesizable vs non-synthesizable constructs.
 - Mealy vs Moore FSM.
 - Record yourself explaining for 5–10 minutes.
 - Output: Self-review notes.
-

4. Phase 2 (Days 16–30): Synthesis & Timing

Objectives

- Understand RTL → Gate-level flow.
- Practice Vivado/Yosys synthesis.
- Learn basic timing (setup/hold) and constraints.

Detailed Day-by-Day

Day 16 – Tool Introduction (Yosys/Vivado)

- Install Vivado WebPACK (if possible) or use Yosys + GTKWave.
- Synthesize: 4-bit adder.
- Output: Area/timing report screenshot.

Day 17 – Synthesis Exploration

- Change coding style (behavioral vs dataflow); re-synthesize.
- Compare timing and area differences.
- Output: Short note: “How coding style affects synthesis”.

Day 18 – VLSI Design Flow Overview

- Study: RTL, synthesis, floorplan, place & route, STA, sign-off (high level).
- Source: Any proper VLSI flow article/video.
- Output: One-page flow diagram in your notebook.

Day 19 – Timing Basics 1

- Learn: Setup time, hold time, clock-to-Q, slack.
- Output: Draw timing diagrams showing setup & hold violation.

Day 20 – Timing Basics 2 (Critical Path)

- For your 4-bit adder or ALU, identify critical path from synthesis report.
- Output: Note critical path and delay.

Day 21 – Lab Expt (Memory)

- Code: Simple RAM (e.g., 16x8) with read/write.
- Testbench: Write/read multiple locations.
- Output: Waveforms saved.

Day 22 – Constraints (SDC/XDC)

- Add clock constraint (e.g., 50 MHz) in tool.
- Re-run timing; see if violations occur.
- Output: Observations on meeting or failing timing.

Day 23 – Low-Power Basics

- Read about: Clock gating, power vs performance trade-offs.
- Add enable-based clock gating to a counter.
- Output: Modified code and note on power saving.

Day 24 – Mini-Project: Enhanced ALU

- 8-bit ALU with operations: add, sub, and, or, xor, compare.
- Testbench + synthesis.
- Output: Project folder in GitHub.

Day 25 – Weekly Review

- Summarize:
 - What is synthesis?
 - What reports you read?
- Output: WEEK2_SUMMARY.md.

Day 26 – UART Basics

- Study UART protocol: baud rate, start/stop bits, parity.
- Output: Handwritten timing diagram for TX and RX.

Day 27 – UART TX Implementation

- Code: UART transmitter in Verilog.

- Testbench: send a few bytes.
- Output: Simulation results + commit.

Day 28 – UART RX / Integration Prep

- Start/outline receiver module (even if not fully complete).
- Output: Partial code + TODO comments.

Day 29 – Clean-Up & Documentation

- Clean code comments, add block diagrams (simple) to README.
- Output: Updated GitHub with diagrams.

Day 30 – Phase 2 Mock Interview

- Practice explaining:
 - Synthesis
 - Setup/hold
 - Your ALU/UART design choices
- Output: Self-assessment notes (what you cannot explain clearly).

5. Phase 3 (Days 31–45): Advanced RTL & Mini Projects

Objectives

- Build 1–2 strong mini-projects.
- Connect designs to real-world systems (IoT/communication).

Day-by-Day Highlights

Day 31–32 – CMOS Basics (Theory)

- Study CMOS inverter, noise margins, switching threshold.
- Output: Plot of VTC (even if approximate) in notebook.

Day 33–35 – Project 1: Digital Stopwatch

- Design FSM for start/stop/reset.
- Implement counter + display logic (assume 7-seg or simple binary).
- Output: Complete Verilog + testbench + GIF/screenshot.

Day 36–38 – Project 2: Sequence Detector + UVM-Friendly

- Refine prior sequence detector: parameterize sequence, add cleaner testbench.
- Output: Reusable testbench architecture.

Day 39–41 – AXI-lite Concepts (High Level)

- Study simple AXI-lite interface; you don't need full implementation, just basic signals.
- Output: Short note + simplified Verilog skeleton.

Day 42–44 – Lab Preparation & Documentation

- Re-run every VTU VLSI lab-style experiment you have done.
- For each: store Verilog file, testbench, waveform screenshot, and a short explanation.
- Output: A single “VLSI_LAB_SUMMARY.pdf” (you can make it in Word/Docs then export).

Day 45 – Week 3 Review

- Ensure 2 mini-projects are fully done and in GitHub.
- Output: WEEK3_SUMMARY.md.

6. Phase 4 (Days 46–60): Verification & STA

Objectives

- Learn basic **SystemVerilog** features for verification.
- Understand the idea of **UVM** (even if you don't master it fully).
- Practice more **timing analysis**.

Day-by-Day Highlights

Day 46–48 – SystemVerilog Intro

- Learn: logic, always_ff, always_comb, simple assertions.
- Apply to your counter/ALU design.
- Output: Updated modules using SystemVerilog style.

Day 49–51 – Assertions & Coverage

- Add a few simple assertions (e.g., illegal state detection) for FSM.
- Track basic functional coverage manually (list of scenarios).
- Output: Assertion files + notes.

Day 52–54 – STA Tool Practice (OpenSTA or similar)

- Try running open-source STA (if possible) or study sample reports.
- Identify paths with negative slack, understand reasons.
- Output: Short write-up on one timing violation and how to fix it.

Day 55–57 – Project 3: SPI Master

- Study SPI protocol: clock, MOSI, MISO, CS.

- Implement basic SPI master module with testbench.
- Output: Complete Verilog + simulation.

Day 58–60 – Python Automation

- Write a small Python script to parse your simulation log or waveform data (even basic).
 - Output: Script in repo, shows simple automation skill.
-

7. Phase 5 (Days 61–75): Backend Intro & Portfolio

Objectives

- Get a basic feel of back-end (physical design).
- Polish resume, GitHub, and LinkedIn as per industry expectations.

Day-by-Day Highlights

Day 61–63 – Physical Design Basics

- Learn: Floorplan, placement, CTS, routing (concept; no need to be expert).
- Output: 1–2 page summary in your own words.

Day 64–66 – Low-Power & Multi-Vt Concepts

- Study clock gating, power vs frequency, multi-threshold cells (theory).
- Output: Add a low-power note to your ALU/UART designs.

Day 67–69 – Portfolio Cleanup

- Ensure:
 - 3 solid projects (ALU, UART, Stopwatch / SPI)
 - Each has: description, block diagram, how to run, and learning outcomes.
- Output: Updated README.md for each project.

Day 70–72 – Resume & LinkedIn

- Create 1-page resume highlighting:
 - VLSI skills, tools (Verilog, Vivado, Icarus, Yosys)
 - 3 projects with bullet points
 - CGPA and relevant coursework.
- Update LinkedIn: photo, headline: “VLSI & Embedded Systems | VTU ECE 2027”.
- Output: PDF resume and LinkedIn URL.

Day 73–75 – Aptitude & DSA (Basic)

- Practice 10–15 aptitude questions/day (quant + logical).

- Do 1–2 DSA questions/day (arrays, strings) from LeetCode or similar.
 - Output: List of solved problems.
-

8. Phase 6 (Days 76–90): Interviews & Applications

Objectives

- Become comfortable with **technical interviews** (core + HR).
- Start serious **off-campus** applications.

Day-by-Day Highlights

Day 76–80 – Technical Mock Interviews

- Each day:
 - Pick 5–10 VLSI interview questions (CMOS, STA, Verilog).
 - Write answers and speak them aloud.
- Once or twice, ask a friend/junior/senior to question you.
- Output: A document with common questions & your model answers.

Day 81–85 – Aptitude + HR Prep

- 20 aptitude questions/day (IndiaBIX).
- Prepare HR answers: “Tell me about yourself”, “Why VLSI?”, “Strengths/weaknesses”.
- Output: HR answer script.

Day 86–88 – Applications

- Target: 15–20 applications per day.
- Platforms: LinkedIn, Naukri, HCL/Intel/Synopsys/Qualcomm career pages, Internshala for internships.
- Output: An Excel/Sheet tracking: company, role, date applied, status.

Day 89–90 – Final Review & Plan Ahead

- Review everything done in 90 days.
- Identify top 3 weak areas; plan next 90 days around them.
- Consider registering for NPTEL/VLSI certificate exam.